

KrishiSaathi

Local Setup Guide

This guide is made for a complete beginner. If you follow each step slowly, you can run this project on your own computer even if you are not an expert.

1) What this project is

KrishiSaathi is an AI-powered farming app for farmers. It helps with:

- Crop recommendation
- Fertilizer recommendation
- Plant disease detection from images
- Weather advisory based on location
- AI farming assistant chatbot
- Multi-language support
- Voice and speech support
- User login and registration

It has:

- Frontend: React app
- Backend: FastAPI Python app
- Database: MongoDB
- AI models: machine learning and Gemini-based chat

2) What you need before starting

You need these installed on your computer:

Required

- Git
- Python 3.10 or 3.11
- Node.js 18 or 20
- npm
- MongoDB 7 OR Docker Desktop
- A browser like Chrome or Edge

Optional but recommended

VS Code editor
Postman (for testing API only)
Docker Desktop for easier MongoDB setup
OpenWeather API key
Gemini API key

Very simple checklist

If you can open terminal and type:

```
git --version  
python --version  
node --version  
npm --version
```

and all show version numbers, you are ready.

3) Install the required software

A. Install Git

Windows

Download Git from: <https://git-scm.com/>
Install it
Keep all default options

macOS

```
brew install git
```

Linux

```
sudo apt update  
sudo apt install git
```

B. Install Python

Windows

Download Python 3.11 from python.org

During install, check: "Add Python to PATH"

macOS

```
brew install python@3.11
```

Linux

```
sudo apt update
```

```
sudo apt install python3 python3-venv python3-pip
```

C. Install Node.js and npm

Windows

Download Node.js 20 LTS from nodejs.org

Install it

macOS

```
brew install node
```

Linux

```
sudo apt install nodejs npm
```

D. Install MongoDB or Docker

Option 1: Use Docker (EASIEST)

Install Docker Desktop, then run:

```
docker --version
```

If Docker works, you can use the project's docker-compose file.

Option 2: Install MongoDB locally

Windows: install MongoDB Community Server

macOS: `brew install mongodb-community@7``

Linux: install MongoDB 7 from official docs

4) Download the project

Open your terminal and run:

```
git clone https://github.com/RakeshYemineni1/Farmer_Companion_FYP.git
cd Farmer_Companion_FYP
cd KrishiSaathi
```

Important: the real repo folder is inside the project folder as `KrishiSaathi`.

5) Open the project folder

After cloning, your project should look like:

```
Farmer_Companion_FYP/
├── KrishiSaathi/
│   ├── backend/
│   ├── frontend/
│   ├── src/
│   ├── .env.example
│   ├── docker-compose.yml
│   ├── requirements.txt
│   └── README.md
```

Open this folder in VS Code or Terminal.

6) Create the Python environment

Inside the project root (`KrishiSaathi`):

Windows (PowerShell)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Windows (Command Prompt)

```
python -m venv venv  
venv\Scripts\activate.bat
```

macOS / Linux

```
python3 -m venv venv  
source venv/bin/activate
```

You should now see `(venv)` in your terminal.

7) Install Python dependencies

Inside the activated virtual environment, run:

```
pip install --upgrade pip  
pip install -r backend/requirements.txt
```

If the project gives an error because of older packages, install the main requirements file too:

```
pip install -r requirements.txt
```

This app needs packages for:

- FastAPI
- MongoDB driver
- TensorFlow / machine learning
- PyTorch and transformers
- OpenWeather + Gemini API access
- image processing and NLP

8) Install frontend dependencies

Now open a second terminal and go to the frontend folder:

```
cd KrishiSaathi/frontend  
npm install
```

This installs React, Tailwind, Axios, and other frontend packages.

9) Set up environment variables

Copy the example file:

```
cp .env.example .env
```

On Windows PowerShell:

```
Copy-Item .env.example .env
```

Now edit the `.env` file.

Example content:

```
MONGODB_URL=mongodb://localhost:27017/krishisaathi
JWT_SECRET_KEY=your_super_secret_key
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
GEMINI_API_KEY=your_gemini_key_here
GROK_API_KEY=your_gemini_key_here
OPENROUTER_API_KEY=your_gemini_key_here
OPENWEATHER_API_KEY=your_openweather_key_here
MODELS_DIR=backend/models/ml_models
REACT_APP_API_BASE_URL=http://localhost:8000
```

What these mean

- ``MONGODB_URL``: where MongoDB is running
- ``JWT_SECRET_KEY``: used for login security
- ``GEMINI_API_KEY``: used for chat and advisory AI
- ``OPENWEATHER_API_KEY``: used for weather forecast
- ``REACT_APP_API_BASE_URL``: tells frontend where the backend lives

Important note

If you do not add API keys, some AI features may still load partially, but some features may not work fully.

10) Start MongoDB

Option A: Use Docker (recommended)

From the project root:

```
docker compose up -d mongo
```

This starts MongoDB on port 27017.

Option B: Use local MongoDB installation

Make sure MongoDB is running, then the app can connect with:

```
mongodb://localhost:27017/krishisaathi
```

To test MongoDB:

```
mongosh
```

Then check:

```
show dbs
```

```
---
```

11) Start the backend server

Start from project root

Make sure your virtual environment is active.

```
cd KrishiSaathi
```

```
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

If you prefer the root-level app:

```
uvicorn src.api.main:app --host 0.0.0.0 --port 8000 --reload
```

What to expect

You should see something like:

Uvicorn running on http://0.0.0.0:8000

Open this in browser:

`http://localhost:8000/docs`

This opens FastAPI Swagger UI.

12) Start the frontend

Open a new terminal and run:

```
cd KrishiSaathi/frontend
```

```
npm start
```

Or:

```
npm run dev
```

The frontend usually runs at:

`http://localhost:3000`

13) Open the app in browser

Open:

`http://localhost:3000`

You will see the KrishiSaathi landing page.

14) How to use each feature

Feature 1: Register and login

1. Click Register or Login
2. Enter your username, email, and password
3. Choose your preferred language
4. Click submit

5. You will enter the dashboard

Note:

In some versions, the app uses demo login and not a full database flow, but it still behaves like a real app for local testing.

Feature 2: Change language

1. Look at the top-right corner
2. Select a language from the dropdown
3. The app updates labels and text
4. The app tries to translate content to the chosen language

Supported languages include:

English
Hindi
Bengali
Telugu
Tamil
Marathi
Kannada
Malayalam
Gujarati
Punjabi

Feature 3: Crop recommendation

1. Click the Crop Recommendation tab
2. Fill in:
 - Nitrogen (N)
 - Phosphorus (P)
 - Potassium (K)
 - Temperature
 - Humidity
 - pH
 - Rainfall

3. Click Get Recommendation
4. The AI model predicts the best crop
5. You see crop name and confidence score

This feature is for farmers deciding which crop is best for a field.

Feature 4: Fertilizer recommendation

1. Click Fertilizer Recommendation
2. Fill in:
 - Temperature
 - Humidity
 - Moisture
 - Soil type
 - Crop type
 - N, P, K values
3. Press submit
4. The app suggests the best fertilizer

This helps reduce wrong fertilizer use and improve soil health.

Feature 5: Disease detection

1. Click Disease Detection
2. Upload a photo of a plant leaf or crop disease image
3. Press submit
4. The model predicts whether the plant is healthy or diseased
5. The result includes disease type and confidence

Best practice:

Use a clear close-up photo

Use proper daylight

Avoid blurry or shadowed images

Feature 6: Weather advisory

1. Click Weather Advisory
2. Allow location access in your browser
3. The app gets your live location
4. It fetches weather forecast data
5. It shows a 7-day forecast
6. The AI gives farming advice based on the weather

This is helpful for:

irrigation planning

rain prediction

frost protection

crop stress alerts

Feature 7: AI chatbot

1. Click Chat
2. Ask questions like:
 - How to improve soil health?
 - Which crop is suitable for rainy season?
 - How do I control pests?
 - What fertilizer should I use?
3. You can also attach an image if needed
4. The chatbot responds in the selected language
5. Some versions support voice input and speech output

The chatbot uses Gemini-style fallback models and farming prompts.

Feature 8: Speech / voice support

The app includes voice input and voice output features in the frontend chat.

How to use:

1. Open the chatbot
2. Choose a language
3. Click the microphone button

4. Speak your question
5. The system reads the spoken words
6. The AI replies
7. The app can read the response aloud

This is useful for farmers who prefer speaking instead of typing.

Feature 9: Results and insights

After each prediction, the app shows:

- prediction result
- confidence percentage
- recommended action
- farm guidance

You can use this information to make decisions before applying fertilizer or planting crops.

15) Common problems and fixes

Problem: MongoDB connection error

Fix:

- Check if MongoDB is running
- Make sure ``MONGODB_URL`` is correct
- Confirm ``docker compose up -d mongo`` ran successfully

Problem: Frontend cannot connect to backend

Fix:

- Check backend is running on port 8000
- Confirm ``.env`` has ``REACT_APP_API_BASE_URL=http://localhost:8000``
- Restart both frontend and backend

Problem: ``npm install`` fails

Fix:

Update npm

Delete `node\_modules` and `package-lock.json` then install again

```
rm -rf node_modules package-lock.json
```

```
npm install
```

On Windows PowerShell:

```
Remove-Item -Recurse -Force node_modules, package-lock.json
```

```
npm install
```

Problem: Python package install fails

Fix:

```
python -m pip install --upgrade pip
```

```
pip install setuptools wheel
```

Then install requirements again.

Problem: API keys missing

Fix:

Add a valid Gemini key

Add a valid OpenWeather key

Restart backend after editing `.env`

16) Quick start summary

If you want the fastest working order:

Terminal 1

```
cd KrishiSaathi
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r backend/requirements.txt
```

```
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

Terminal 2

```
cd KrishiSaathi/frontend
```

```
npm install
```

```
npm start
```

Terminal 3 (optional for MongoDB through Docker)

```
cd KrishiSaathi
```

```
docker compose up -d mongo
```

Then open:

```
http://localhost:3000
```

17) Recommended operating system notes

Windows

Use Git Bash or PowerShell. Use:

```
venv\Scripts\activate
```

macOS

Use Terminal and:

```
source venv/bin/activate
```

Linux

Use Terminal and:

```
source venv/bin/activate
```

18) Important final advice

Always activate the Python virtual environment before running backend commands

Keep the frontend and backend running in separate terminals

Keep MongoDB running before backend start

Do not worry if the first run shows small errors; most are due to missing environment variables or missing MongoDB

Use the ``.env`` file to set API keys and database connection

19) Final success check

Your project is running correctly when:

MongoDB is active

Backend is running on ``http://localhost:8000``

Frontend is running on ``http://localhost:3000``

Login page shows correctly

Crop/fertilizer/disease/weather/chat features work without breaking

20) One-sentence summary

KrishiSaathi is a complete AI farming assistant that can recommend crops, suggest fertilizers, detect diseases from images, analyze weather, and answer agriculture questions in multiple languages.

If you want the easiest local setup, use:

1. Python virtual environment
2. MongoDB via Docker
3. Backend with uvicorn
4. Frontend with npm start
5. ``.env`` with Gemini + OpenWeather keys

And you are done.